Based on the Group Discussion on Friday (20/03), the following aspects/features were discussed for R&D for the group programming assignment for a Workout Planner/Gym Activity tracker.

Project Brief - Potential List of Project Requirements:

- Tracking exercise – sets, rep, workout length/frequency, running times (long-term tracking long-term data tracking/presenting), PR Tracking
- Training Schedules/Plan – list of exercises for a given day
- Calories burned (Active), eaten, basal metabolic rate - caloric guide based on user goal (bulk/cut/maintain)
- Nutrition tracking – macros breakdown/target goal driven ○ User notification/updates provided regularly

- AI Aspects: ○ Food calorie estimation/macros breakdown based on user image
    - ○ Calculating future reps based on past exercise ○ Checking form based on videos
    - ○ Body progress comparison

- Integrating fitness data collection with system health applications such as Apple Health and Samsung Health

- Extension – implementing a social media side (adding friends, seeing their workout/activities – if rest finished early)

Research on current workout/fitness tracker applications:

Current Applications	Key Features Implemented
Hevy, Strong	• Track activities (sets and reps) for workouts • Creating training schedules • Providing long-term progress data • Tracking PR • Instructions/animations shows proper form/technique • Integrates data to system health platform
Strava	• Implements the social aspect well – tracking and posting activities, adding friends, facilitating the integration of posts directly to main social media platforms such as Instagram

MyFitnessPal	• Track food intakes – macros breakdown
	• Custom meals can be added and saved • Ease of use - Scanning barcode of foods to add or manually searching for them • Integrates with the system health platform to deliver accurate caloric goals based on the daily activity and users goal

Key findings: *Existing apps cover the features discussed, but no single app does all of them well. Fitness trackers like Strong and Hevy lack nutrition logging; calorie trackers like MyFitnessPal lack meaningful activity tracking. Our app aims to bridge this gap with a comprehensive solution covering both.*

Researching System Integration and AI LLM Implementation (through Java):

- **Integration – Apple health/Samsung health** ○ Access to Apple health based on user permission; implemented through HealthKit framework in Swift/objective C ○ Access to Samsung health data through Samsung Health Data SDK; partnership needs to be requested and approved by Samsung prior; based on user permission

- **Selecting LLM model – cost, implementation, usability, limitations** ○ Usage driven thru API Calls ○ Current Implementations:
  - Fitness Companies purchasing API keys through LLM deployment companies i.e OpenAI, Anthropic
  - Cost negotiated based on usage
  - Companies also utilising 3rd party aggregators such as OpenRouter (multiple AI model access through one API key)
  - ○ LLM models to consider for the project – free model whilst being accurate:
    - Utilising OpenRouter – has multiple free models which can be implemented
    - Selection can be easily modulated in code

Input	Output	Best model (based on accuracy)
Text	Text	NVIDIA: Nemotron 3 Super
Image	Text	NVIDIA: Nemotron Nano 12B 2 VL

Image + Text	Image	NVIDIA: Nemotron Nano 12B 2 VL

- Implementing OpenRouter in Java
- Based on current implementations, **Spring Boot framework** has the widest usage and most support across the most API interactions with LLMs ○ Framework handles routing, config, and dependency injection – only AI logic needs to be implemented
    - Spring AI is the key library — provides a unified interface to OpenAI, Anthropic, Ollama, and more ○ Swap LLM providers by changing config only, no code refactoring needed
    - ChatClient is the main class for sending prompts and receiving responses
    - Supports RAG (Retrieval-Augmented Generation) via built-in vector store integration
    - Prompt templates keep prompt engineering separate from Java code
    - API keys and model settings managed through application.properties
    - Sits at the engineering/deployment layer above the model, not involved in how the LLM thinks

To better understand the implementation of Spring Boot framework, a test SpringBootApplication was developed (see GitHub repo) to perform the following:

- Used Java Spring Boot to connect to and query large language models (LLMs) via the OpenRouter API
- Accessed multiple AI models (Google, Anthropic, Meta, OpenAI) through a single API key
- Configured model behaviour using system prompts to control response format and style
- Switched between different LLMs by changing a single model name string in code
- Sent image files to vision-capable models and received text descriptions in return
- Used Spring AI library to simplify API communication, handling authentication and request formatting automatically
- Tested API endpoints locally using curl commands in the terminal